Міністерство освіти і науки України
Національний університет «Одеська політехніка»
Навчально-науковий інститут комп’ютерних систем
Кафедра інформаційних систем

Лабораторна робота № 3
з дисципліни: «Теорія алгоритмів»
Тема: «Спеціалізована структура даних *Heap* як основа ефективного нестійкого сортування»

Варіант № 15

Виконав:
Студент групи AI-254
Панькевич В. В.
Перевірили:
Смик С. Ю.
Арсірій О.О.

Одеса 2026

Мета роботи: набуття практичних навичок із проектування, реалізації, тестування та аналізу алгоритму сортування купою (Heapsort). За рахунок виконання трасування та візуалізації у вигляді графа практично довести логарифмічну часову складність та нестійкість алгоритму сортування купою, а також порівняти цей алгоритм за кількістю базових операцій з алгоритмом швидкого сортування.

Завдання:

1. Спроектувати, реалізувати, протестувати та проаналізувати алгоритм сортування купою.
2. Виконати трасування та візуалізацію 2-х фаз алгоритму сортування купою, надати пояснення.
3. Довести на прикладі, що алгоритм сортування купою є нестійким та порівняти обчислювальну складність алгоритмів сортування купою та швидкого сортування за базовими операціями.

Варіант	Послідовність
15	50, 80, 19, 86, 35, 7, 60, 48, 51

Результати виконання завдання:

1. Сортування купою (HeapSort)

Для реалізації алгоритму сортування купою використовується структура даних максимальна купа (Max-Heap), у якій значення кожного батьківського вузла більше або дорівнює значенням його дочірніх вузлів. Завдяки цьому найбільший елемент завжди знаходиться в корені купи.

Алгоритм складається з двох основних фаз:

1) Побудова максимальної купи – перетворення початкового масиву на структуру Max-Heap.

2) Сортування – послідовне переміщення найбільшого елемента в кінець масиву та відновлення властивостей купи за допомогою процедури Sink.

Алгоритм побудови максимальної купи реалізується за допомогою процедури занурення Sink, яка забезпечує виконання властивостей максимальної купи. Псевдокод алгоритму Heap_Max та допоміжної процедури Sink наведено в таблиці 3.1.

Псевдокод алгоритму сортування купою HeapSort, який складається з фаз побудови максимальної купи та сортування елементів масиву, наведено в таблиці 3.2.

Таблиця 3.1 – Приклад псевдокоду алгоритму створення максимальної купи на базі процедури занурення

```
Heap_Max(A)
// Вхідні дані: невідсортований масив A
// Вихідні дані: масив A, який є максимальною купою
   n ← |A|
   // Починаємо з першого елемента, що має дочірні вузли та рухаємося до початку масиву
   for i ← floor(n / 2) – 1 down to 0 do
       Sink(A, i, n)
```

```
Sink(A, i, n)
// Вхідні дані: масив A, індекс i, розмір купи n
// n – розмір купи
// Вихідні дані: масив A з відновленою властивістю купи

   k ← i
   while True do
       j ← 2 * k + 1 // Індекс лівого дочірнього елемента

       // Перевіряємо, чи існує лівий дочірній елемент
       if j >= n then
           break

       // Знаходимо індекс найбільшого дочірнього елемента
       if j + 1 < n and A[j + 1] > A[j] then
           j ← j + 1
            // Порівнюємо поточний елемент з найбільшим дочірнім
       if A[k] >= A[j] then
           break // Елемент на своєму місці, вихід з циклу
       else
           Swap(A[k], A[j])
           k ← j
```

Таблиця 3.2 – Приклад псевдокоду алгоритму сортування купою на базі процедури занурення Sink(A, i, n)

```
HeapSort(A)
// Вхідні дані: невідсортований масив A
// Вихідні дані: відсортований масив A
    n ← |A|
   // Фаза 1: Побудова максимальної купи
   // Починаємо з першого елемента, що має дочірні вузли, і рухаємося до початку
масиву
for i ← floor(n / 2) – 1 down to 0 do
    Sink(A, i, n)

   // Фаза 2: Сортування
   // Повторно вилучаємо максимальний елемент і переміщуємо його в кінець масиву
for i ← n – 1 down to 1 do
    Swap(A[0], A[i])
   // Відновлюємо властивості купи для зменшеного масиву.
    Sink(A, 0, i)
```

Для практичної реалізації алгоритму сортування купою було розроблено програму мовою Python, яка виконує побудову максимальної купи та подальше впорядкування її елементів. Програма також передбачає підрахунок базових операцій для подальшого аналізу ефективності алгоритму. Відповідний програмний код наведено в Лістингу 3.1.

Лістинг 3.1 – Python-код реалізації алгоритму сортування купою

```
def swap(arr, i, j):
    """Міняє місцями два елементи масиву."""
    arr[i], arr[j] = arr[j], arr[i]
    return 3  # Кількість присвоювань при обміні


def sink(arr, i, n):
    """
    Процедура занурення елемента вниз по купі.
    arr: масив
    i: індекс поточного елемента
    n: розмір купи
    """
    comparisons = 0
    assignments = 0
    k = i

    while True:
        j = 2 * k + 1  # Індекс лівого дочірнього елемента

        if j >= n:
            break

        # Знаходимо найбільший дочірній елемент
        if j + 1 < n:
            comparisons += 1
            if arr[j + 1] > arr[j]:
                j += 1

        # Порівнюємо батьківський елемент із дочірнім
        comparisons += 1
```

```
        if arr[k] >= arr[j]:
            break

        # Обмінюємо елементи та продовжуємо занурення
        assignments += swap(arr, k, j)
        k = j

    return comparisons, assignments


def heapsort(arr):
    """
    Алгоритм сортування купою (HeapSort).
    """
    n = len(arr)
    comparisons = 0
    assignments = 0

    print("Початковий масив:", arr)

    # Фаза 1: Побудова максимальної купи
    print("\n--- Фаза 1: Побудова максимальної купи ---")

    for i in range(n // 2 - 1, -1, -1):
        print(f"Занурюємо елемент з індексу {i}: {arr[i]}")

        c, a = sink(arr, i, n)
        comparisons += c
        assignments += a

    print("Масив після побудови купи:", arr)

    # Фаза 2: Сортування
    print("\n--- Фаза 2: Сортування ---")

    for i in range(n - 1, 0, -1):
        print(f"\nМіняємо місцями корінь ({arr[0]}) "
              f"та останній елемент ({arr[i]})")

        assignments += swap(arr, 0, i)

        # Відновлюємо властивості зменшеної купи
        print(f"Розмір купи зменшився до {i}.")

        c, a = sink(arr, 0, i)
        comparisons += c
        assignments += a

        print("Масив на поточному кроці:", arr)

    return arr, comparisons, assignments


# Моделювання для варіанта №15
A = [50, 80, 19, 86, 35, 7, 60, 48, 51]

sorted_A, comparisons, assignments = heapsort(A)

print("\n Відсортований масив:", sorted_A)
print("Загальна кількість порівнянь:", comparisons)
print("Загальна кількість присвоювань:", assignments)
```

Результати виконання Python-коду, наведеного в Лістингу 3.1, представлено в таблиці 3.3.

Таблиця 3.3 – Результати виконання алгоритму сортування купою

```
Початковий масив: [50, 80, 19, 86, 35, 7, 60, 48, 51]

--- Фаза 1: Побудова максимальної купи ---
Занурюємо елемент з індексу 3: 86
Занурюємо елемент з індексу 2: 19
Занурюємо елемент з індексу 1: 80
Занурюємо елемент з індексу 0: 50
Масив після побудови купи: [86, 80, 60, 51, 35, 7, 19, 48, 50]

--- Фаза 2: Сортування ---

Міняємо місцями корінь (86) та останній елемент (50)
Розмір купи зменшився до 8.
Масив на поточному кроці: [80, 51, 60, 50, 35, 7, 19, 48, 86]

Міняємо місцями корінь (80) та останній елемент (48)
Розмір купи зменшився до 7.
Масив на поточному кроці: [60, 51, 48, 50, 35, 7, 19, 80, 86]

Міняємо місцями корінь (60) та останній елемент (19)
Розмір купи зменшився до 6.
Масив на поточному кроці: [51, 50, 48, 19, 35, 7, 60, 80, 86]

Міняємо місцями корінь (51) та останній елемент (7)
Розмір купи зменшився до 5.
Масив на поточному кроці: [50, 35, 48, 19, 7, 51, 60, 80, 86]

Міняємо місцями корінь (50) та останній елемент (7)
Розмір купи зменшився до 4.
Масив на поточному кроці: [48, 35, 7, 19, 50, 51, 60, 80, 86]

Міняємо місцями корінь (48) та останній елемент (19)
Розмір купи зменшився до 3.
Масив на поточному кроці: [35, 19, 7, 48, 50, 51, 60, 80, 86]

Міняємо місцями корінь (35) та останній елемент (7)
Розмір купи зменшився до 2.
Масив на поточному кроці: [19, 7, 35, 48, 50, 51, 60, 80, 86]

Міняємо місцями корінь (19) та останній елемент (7)
Розмір купи зменшився до 1.
Масив на поточному кроці: [7, 19, 35, 48, 50, 51, 60, 80, 86]

Відсортований масив: [7, 19, 35, 48, 50, 51, 60, 80, 86]
Загальна кількість порівнянь: 36
Загальна кількість присвоювань: 69
```

2. Трасування та візуалізація алгоритму сортування купою

Для детального дослідження роботи алгоритму сортування купою програмний код Лістингу 3.1 було доповнено командами виведення проміжних результатів. Це дозволяє простежити процес порівняння та перестановки елементів під час побудови максимальної купи й подальшого сортування масиву. Відповідний Python-код трасування наведено в Лістингу 3.2.

Результати трасування обох фаз алгоритму HeapSort для варіанта №15 представлено в таблиці 3.4.

Лістинг 3.2 – Python-код реалізації алгоритму сортування купою (трасування)

```
def swap(arr, i, j):
    """Міняє місцями два елементи масиву."""
    arr[i], arr[j] = arr[j], arr[i]
    return 3

def sink(arr, i, n):
    """
    Процедура занурення елемента вниз по купі.
    arr: масив
    i: індекс поточного елемента
    n: розмір купи
    """
    comparisons = 0
    assignments = 0
    k = i

    print(f"Починаємо 'занурювати' елемент: {arr[k]} "
          f"з індексу {k}")

    while True:
        j = 2 * k + 1  # Індекс лівого дочірнього елемента

        if j >= n:
            print(f"Елемент {arr[k]} на індексі {k} "
                  f"досяг кінця купи. Завершуємо.")
            break

        # Знаходимо найбільший дочірній елемент
        if j + 1 < n:
            comparisons += 1
            if arr[j + 1] > arr[j]:
                print(f"  Порівнюємо {arr[j]} (лівий) та "
                      f"{arr[j + 1]} (правий). "
                      f"Обираємо {arr[j + 1]}")
                j += 1
            else:
                print(f"  Порівнюємо {arr[j]} (лівий) та "
                      f"{arr[j + 1]} (правий). "
                      f"Обираємо {arr[j]}")
```

```
        else:
            print(f"  Обираємо лівий дочірній елемент: "
                  f"{arr[j]} на індексі {j}")

        # Порівнюємо батьківський та дочірній елементи
        comparisons += 1
        if arr[k] >= arr[j]:
            print(f"  {arr[k]} (батько) >= {arr[j]} "
                  f"(найбільший дочірній). "
                  f"Елемент на своєму місці.")
            break

        # Обмінюємо елементи
        print(f"  Міняємо місцями {arr[k]} (батько) "
              f"та {arr[j]} (дочірній)")

        assignments += swap(arr, k, j)
        print(f"  Масив після обміну: {arr}")

        k = j

    return comparisons, assignments

def heapsort(arr):
    """
    Алгоритм сортування купою (HeapSort).
    """
    n = len(arr)
    comparisons = 0
    assignments = 0

    print("Початковий масив:", arr)

    # Фаза 1: Побудова максимальної купи
    print("\n--- Фаза 1: Побудова максимальної купи ---")

    for i in range(n // 2 - 1, -1, -1):
        c, a = sink(arr, i, n)
        comparisons += c
        assignments += a
        print("----------------------------------")

    print("\n Масив після побудови купи:", arr)

    # Фаза 2: Сортування
    print("\n--- Фаза 2: Сортування ---")

    for i in range(n - 1, 0, -1):
        print(f"\nМіняємо місцями корінь ({arr[0]}) "
              f"та останній елемент ({arr[i]})")

        assignments += swap(arr, 0, i)

        print(f"Розмір купи зменшився до {i}. "
              f"Відновлюємо властивості купи.")

        c, a = sink(arr, 0, i)
        comparisons += c
        assignments += a

        print("Масив на поточному кроці:", arr)

    return arr, comparisons, assignments
```

```
# Моделювання для варіанта №15
A = [50, 80, 19, 86, 35, 7, 60, 48, 51]

sorted_A, comparisons, assignments = heapsort(A)

print("\n Відсортований масив:", sorted_A)
print("Загальна кількість порівнянь:", comparisons)
print("Загальна кількість присвоювань:", assignments)
```

Таблиця 3.4 – Результати трасування алгоритму сортування купою

```
Початковий масив: [50, 80, 19, 86, 35, 7, 60, 48, 51]

--- Фаза 1: Побудова максимальної купи ---
Починаємо 'занурювати' елемент: 86 з індексу 3
  Порівнюємо 48 (лівий) та 51 (правий). Обираємо 51
  86 (батько) >= 51 (найбільший дочірній). Елемент на своєму місці.
---------------------------------
Починаємо 'занурювати' елемент: 19 з індексу 2
  Порівнюємо 7 (лівий) та 60 (правий). Обираємо 60
  Міняємо місцями 19 (батько) та 60 (дочірній)
  Масив після обміну: [50, 80, 60, 86, 35, 7, 19, 48, 51]
Елемент 19 на індексі 6 досяг кінця купи. Завершуємо.
---------------------------------
Починаємо 'занурювати' елемент: 80 з індексу 1
  Порівнюємо 86 (лівий) та 35 (правий). Обираємо 86
  Міняємо місцями 80 (батько) та 86 (дочірній)
  Масив після обміну: [50, 86, 60, 80, 35, 7, 19, 48, 51]
  Порівнюємо 48 (лівий) та 51 (правий). Обираємо 51
  80 (батько) >= 51 (найбільший дочірній). Елемент на своєму місці.
---------------------------------
Починаємо 'занурювати' елемент: 50 з індексу 0
  Порівнюємо 86 (лівий) та 60 (правий). Обираємо 86
  Міняємо місцями 50 (батько) та 86 (дочірній)
  Масив після обміну: [86, 50, 60, 80, 35, 7, 19, 48, 51]
  Порівнюємо 80 (лівий) та 35 (правий). Обираємо 80
  Міняємо місцями 50 (батько) та 80 (дочірній)
  Масив після обміну: [86, 80, 60, 50, 35, 7, 19, 48, 51]
  Порівнюємо 48 (лівий) та 51 (правий). Обираємо 51
  Міняємо місцями 50 (батько) та 51 (дочірній)
  Масив після обміну: [86, 80, 60, 51, 35, 7, 19, 48, 50]
Елемент 50 на індексі 8 досяг кінця купи. Завершуємо.
---------------------------------

Масив після побудови купи: [86, 80, 60, 51, 35, 7, 19, 48, 50]

--- Фаза 2: Сортування ---

Міняємо місцями корінь (86) та останній елемент (50)
Розмір купи зменшився до 8. Відновлюємо властивості купи.
Починаємо 'занурювати' елемент: 50 з індексу 0
  Порівнюємо 80 (лівий) та 60 (правий). Обираємо 80
  Міняємо місцями 50 (батько) та 80 (дочірній)
  Масив після обміну: [80, 50, 60, 51, 35, 7, 19, 48, 86]
  Порівнюємо 51 (лівий) та 35 (правий). Обираємо 51
  Міняємо місцями 50 (батько) та 51 (дочірній)
  Масив після обміну: [80, 51, 60, 50, 35, 7, 19, 48, 86]
  Обираємо лівий дочірній елемент: 48 на індексі 7
  50 (батько) >= 48 (найбільший дочірній). Елемент на своєму місці.
Масив на поточному кроці: [80, 51, 60, 50, 35, 7, 19, 48, 86]
```

Продовження таблиці 3.4

```
Міняємо місцями корінь (80) та останній елемент (48)
Розмір купи зменшився до 7. Відновлюємо властивості купи.
Починаємо 'занурювати' елемент: 48 з індексу 0
  Порівнюємо 51 (лівий) та 60 (правий). Обираємо 60
  Міняємо місцями 48 (батько) та 60 (дочірній)
  Масив після обміну: [60, 51, 48, 50, 35, 7, 19, 80, 86]
  Порівнюємо 7 (лівий) та 19 (правий). Обираємо 19
  48 (батько) >= 19 (найбільший дочірній). Елемент на своєму місці.
Масив на поточному кроці: [60, 51, 48, 50, 35, 7, 19, 80, 86]

Міняємо місцями корінь (60) та останній елемент (19)
Розмір купи зменшився до 6. Відновлюємо властивості купи.
Починаємо 'занурювати' елемент: 19 з індексу 0
  Порівнюємо 51 (лівий) та 48 (правий). Обираємо 51
  Міняємо місцями 19 (батько) та 51 (дочірній)
  Масив після обміну: [51, 19, 48, 50, 35, 7, 60, 80, 86]
  Порівнюємо 50 (лівий) та 35 (правий). Обираємо 50
  Міняємо місцями 19 (батько) та 50 (дочірній)
  Масив після обміну: [51, 50, 48, 19, 35, 7, 60, 80, 86]
Елемент 19 на індексі 3 досяг кінця купи. Завершуємо.
Масив на поточному кроці: [51, 50, 48, 19, 35, 7, 60, 80, 86]

Міняємо місцями корінь (51) та останній елемент (7)
Розмір купи зменшився до 5. Відновлюємо властивості купи.
Починаємо 'занурювати' елемент: 7 з індексу 0
  Порівнюємо 50 (лівий) та 48 (правий). Обираємо 50
  Міняємо місцями 7 (батько) та 50 (дочірній)
  Масив після обміну: [50, 7, 48, 19, 35, 51, 60, 80, 86]
  Порівнюємо 19 (лівий) та 35 (правий). Обираємо 35
  Міняємо місцями 7 (батько) та 35 (дочірній)
  Масив після обміну: [50, 35, 48, 19, 7, 51, 60, 80, 86]
Елемент 7 на індексі 4 досяг кінця купи. Завершуємо.
Масив на поточному кроці: [50, 35, 48, 19, 7, 51, 60, 80, 86]

Міняємо місцями корінь (50) та останній елемент (7)
Розмір купи зменшився до 4. Відновлюємо властивості купи.
Починаємо 'занурювати' елемент: 7 з індексу 0
  Порівнюємо 35 (лівий) та 48 (правий). Обираємо 48
  Міняємо місцями 7 (батько) та 48 (дочірній)
  Масив після обміну: [48, 35, 7, 19, 50, 51, 60, 80, 86]
Елемент 7 на індексі 2 досяг кінця купи. Завершуємо.
Масив на поточному кроці: [48, 35, 7, 19, 50, 51, 60, 80, 86]

Міняємо місцями корінь (48) та останній елемент (19)
Розмір купи зменшився до 3. Відновлюємо властивості купи.
Починаємо 'занурювати' елемент: 19 з індексу 0
  Порівнюємо 35 (лівий) та 7 (правий). Обираємо 35
  Міняємо місцями 19 (батько) та 35 (дочірній)
  Масив після обміну: [35, 19, 7, 48, 50, 51, 60, 80, 86]
Елемент 19 на індексі 1 досяг кінця купи. Завершуємо.
Масив на поточному кроці: [35, 19, 7, 48, 50, 51, 60, 80, 86]

Міняємо місцями корінь (35) та останній елемент (7)
Розмір купи зменшився до 2. Відновлюємо властивості купи.
Починаємо 'занурювати' елемент: 7 з індексу 0
  Обираємо лівий дочірній елемент: 19 на індексі 1
  Міняємо місцями 7 (батько) та 19 (дочірній)
  Масив після обміну: [19, 7, 35, 48, 50, 51, 60, 80, 86]
Елемент 7 на індексі 1 досяг кінця купи. Завершуємо.
Масив на поточному кроці: [19, 7, 35, 48, 50, 51, 60, 80, 86]
```

Кінець таблиці 3.4

```
Міняємо місцями корінь (19) та останній елемент (7)
Розмір купи зменшився до 1. Відновлюємо властивості купи.
Починаємо 'занурювати' елемент: 7 з індексу 0
Елемент 7 на індексі 0 досяг кінця купи. Завершуємо.
Масив на поточному кроці: [7, 19, 35, 48, 50, 51, 60, 80, 86]

Відсортований масив: [7, 19, 35, 48, 50, 51, 60, 80, 86]
Загальна кількість порівнянь: 36
Загальна кількість присвоювань: 69
```

Для наочного представлення роботи алгоритму сортування купою на основі результатів трасування, наведених у таблиці 3.4, побудовано графічні схеми обох фаз. На рисунку 3.1 показано процес формування максимальної купи, а на рисунку 3.2 – послідовне переміщення найбільших елементів у відсортовану частину масиву.

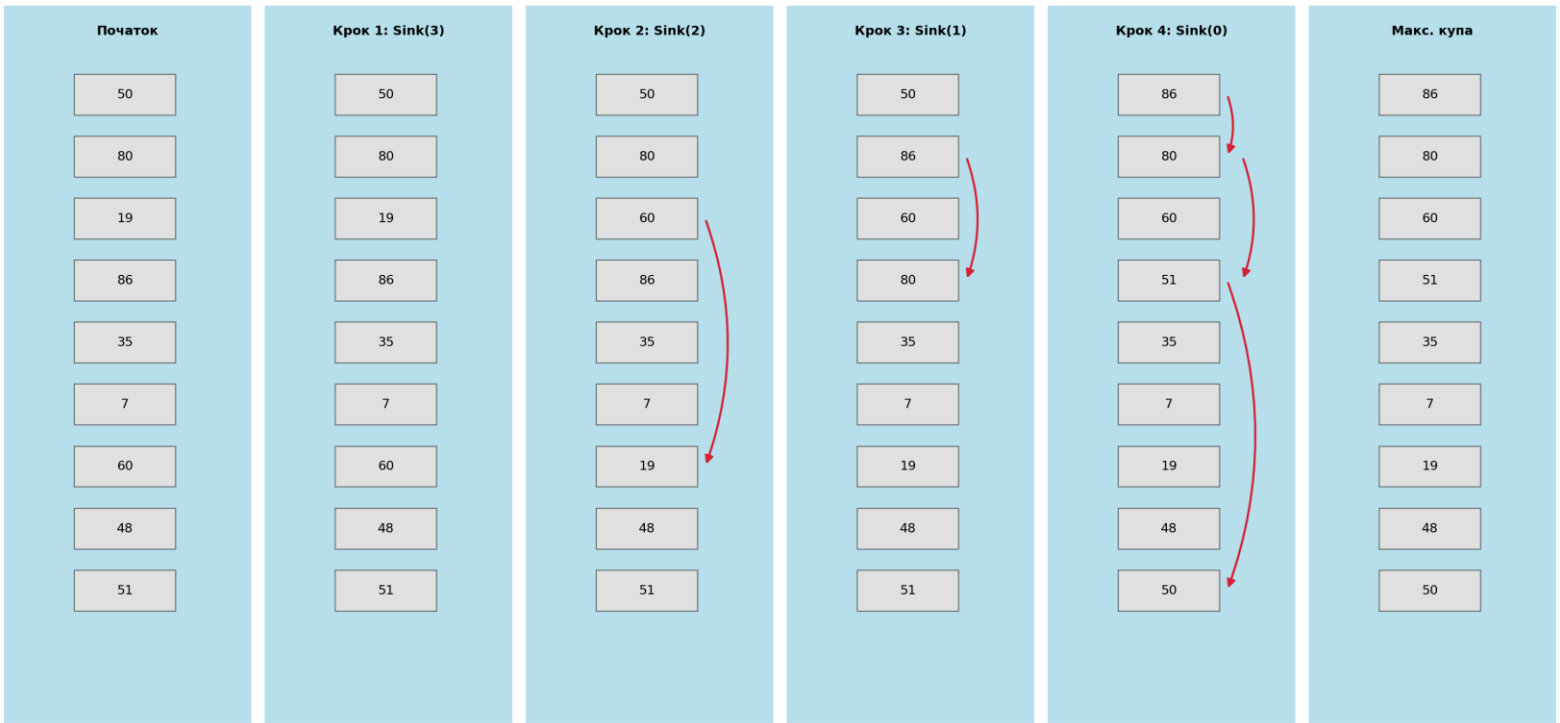


Рисунок 3.1 – Візуалізація кроків фази побудови максимальної купи алгоритму HeapSort

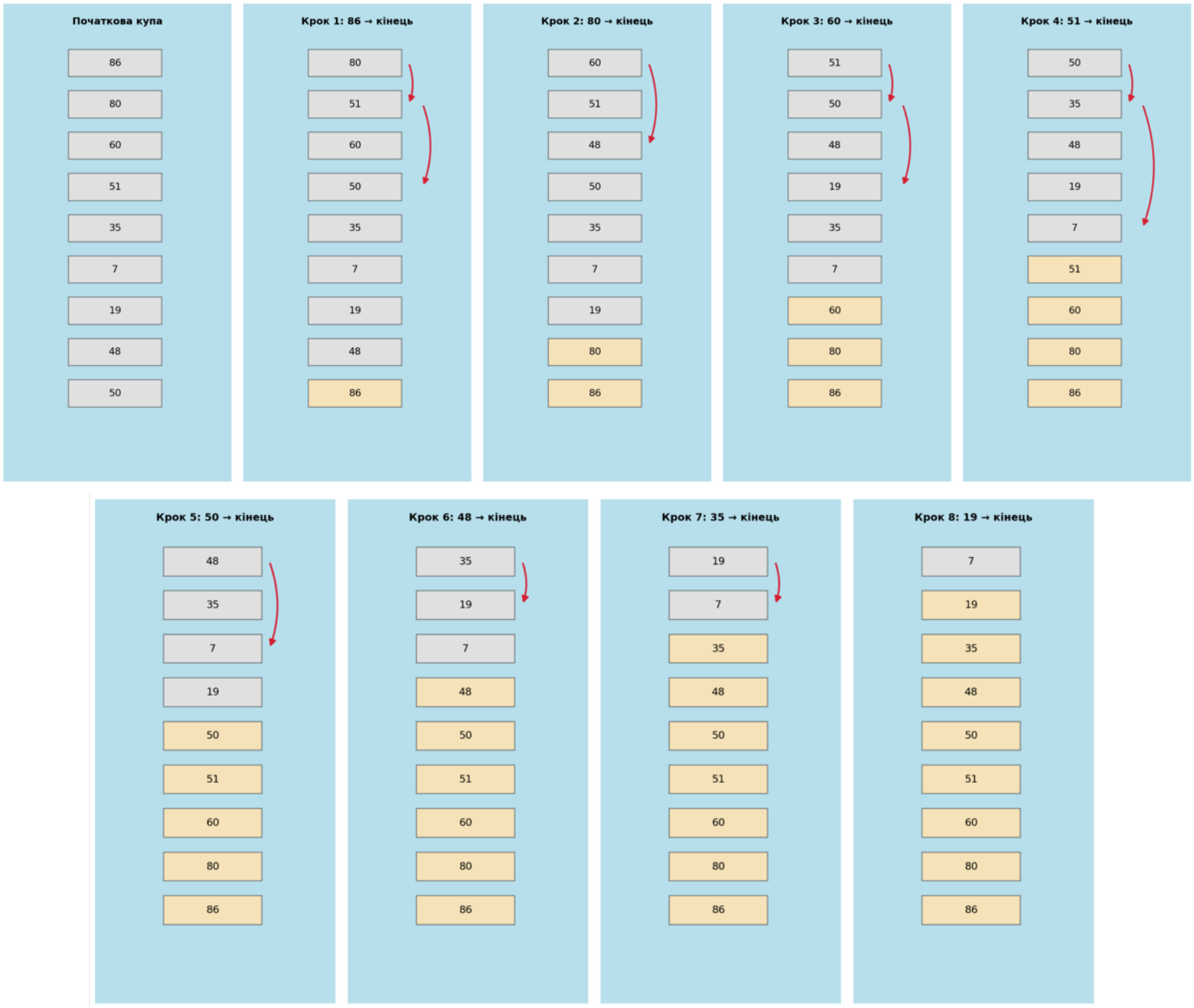


Рисунок 3.2 – Візуалізація кроків фази сортування алгоритму HeapSort

Пояснення результатів трасування та візуалізації:

Аналізуючи результати трасування (таблиця 3.4) та побудовані графічні схеми (рисунки 3.1 і 3.2), можна простежити роботу алгоритму сортування купою. На першій фазі відбувається послідовне занурення елементів, у результаті чого формується максимальна купа з найбільшим елементом 86 у корені. На другій фазі найбільші елементи поступово переміщуються в кінець масиву, а розмір купи зменшується після кожного кроку. Після восьми кроків отримано повністю відсортований масив [7, 19, 35, 48, 50, 51, 60, 80, 86].

3. Аналіз нестійкості та обчислювальної складності алгоритмів сортування

3.1. Доведення нестійкості алгоритму HeapSort

Алгоритм сортування купою HeapSort належить до нестійких алгоритмів, оскільки під час обміну елементів може змінюватися початковий порядок елементів з однаковими ключами. Для демонстрації цієї властивості розглянемо масив із повторюваними значеннями, позначеними різними індексами.

Розглянемо масив:

$[2_1, 2_2, 1]$ - тут 2_1 та 2_2 мають однакове значення, але різні початкові позиції (див. таблицю 3.5).

Таблиця 3.5 – Демонстрація нестійкості алгоритму HeapSort

Етап	Стан масиву
Початковий масив	$[2_1, 2_2, 1]$
Побудова максимальної купи	$[2_1, 2_2, 1]$
Переміщення кореня в кінець	$[1, 2_2, 2_1]$
Відновлення купи	$[2_2, 1, 2_1]$
Завершення сортування	$[1, 2_2, 2_1]$

У початковому масиві 2_1 стоїть перед 2_2, але після сортування їхній порядок змінюється.

Висновок: Отже, у результаті сортування елементи з однаковими значеннями змінили свій початковий порядок. Це підтверджує, що алгоритм HeapSort не забезпечує стійкості сортування.

3.2. Порівняльний аналіз алгоритмів HeapSort та QuickSort

Для порівняння ефективності алгоритмів HeapSort та QuickSort було проаналізовано їхню теоретичну обчислювальну складність і кількість базових операцій для однакової вхідної послідовності варіанта №15. Для коректного

порівняння використовувався єдиний підхід до підрахунку операцій: враховувалися порівняння значень елементів масиву та присвоювання під час їх обміну (3 присвоювання на один обмін). Результати порівняльного аналізу наведено в таблиці 3.6.

Таблиця 3.6 – Порівняння кількості базових операцій алгоритмів сортування

Назва алгоритму	Теоретичні відомості		Результати експерименту n = 9, A = [50, 80, 19, 86, 35, 7, 60, 48, 51]	
	Порівняння	Присвоювання	Порівняння	Присвоювання при обмінах
HeapSort	O(n log n)	O(n log n)	36	69
QuickSort	O(n log n) – середній випадок; $O(n^2)$ – найгірший	O(n log n) – середній випадок; $O(n^2)$ – найгірший	47	27

За результатами таблиці 3.6 встановлено, що алгоритм HeapSort виконав 36 порівнянь та 69 присвоювань, тоді як QuickSort – 47 порівнянь та 27 присвоювань. Таким чином, для заданої послідовності сортування купою потребувало меншої кількості порівнянь, але більшої кількості присвоювань.

Обидва алгоритми мають середню часову складність O(n log n), проте HeapSort зберігає її навіть у найгіршому випадку, тоді як для QuickSort складність може зростати до $O(n^2)$.

Побудова максимальної купи має часову складність O(n), оскільки більшість елементів розташована на нижніх рівнях дерева та потребує невеликої кількості операцій занурення. Під час другої фази виконується n – 1 кроків сортування, на кожному з яких процедура Sink має складність O(log n). Таким чином, загальна часова складність алгоритму HeapSort становить O(n log n).

Висновок:

Під час виконання лабораторної роботи було спроєктовано, реалізовано, протестовано та проаналізовано алгоритм сортування купою (HeapSort) мовою Python. Для вхідної послідовності варіанта №15 було виконано побудову максимальної купи та подальше сортування масиву за зростанням.

За допомогою трасування та графічної візуалізації досліджено обидві фази алгоритму: формування максимальної купи та послідовне переміщення найбільших елементів у кінець масиву. У результаті було отримано відсортовану послідовність [7, 19, 35, 48, 50, 51, 60, 80, 86].

На окремому прикладі продемонстровано нестійкість алгоритму HeapSort. Порівняльний аналіз із QuickSort показав, що для заданої послідовності HeapSort виконав 36 порівнянь та 69 присвоювань при обмінах, тоді як QuickSort – 47 порівнянь та 27 присвоювань. Також встановлено, що перевагою HeapSort є гарантована часова складність O(n log n) навіть у найгіршому випадку, на відміну від QuickSort, складність якого може досягати $O(n^2)$.

Посилання на GitHub: https://github.com/Pankevich-Vyacheslav/Lab_3/tree/main

Посилання на онлайн компілятор: https://coddy.tech/playground/ru/python